

IGNISCORE TALK · 2026

# 바이트 코딩으로 빛어낸 LLM 엔진

---

"해본 적 없는 사람이, 해본 적 없는 방식으로, 해본 적 없는 것  
을 만든다"

.NET 10

VULKAN 1.3

GEMMA 4

VIBE CODING

## 프롤로그

"C# 하나로, 혼자서,  
llama.cpp 같은 걸 **정말로** 만들 수 있을까?"

*IgnisCore* · 바이브 코딩으로 빚어낸 LLM 엔진

— 1년 전, 저는 반신반의했습니다.

## 오늘 드릴 답

가능합니다.

그리고 그 열쇠는 **바이브 코딩**에 있습니다.

- llama.cpp와 **decode 속도 94%** 까지 온 C# 엔진
- 개인 한 명 + 주말 + AI 페어 = **1,853개 GPU 텐서** 움직이는 파이프라인
- 그리고 이 자리에서 나누고 싶은, 당신 차례의 이야기

# 오늘의 네 흐름

## I · 바이브

어려워 보이는 일을  
작게 쪼개 시작한다.

**AI와 함께 걷는 방식.**

## II · 사례

C# + Vulkan으로 만든  
LLM 엔진.

**"된다"는 증거.**

## III · 존중

그럼에도 llama.cpp는  
위대하다.

**전문성은 사라지지 않는다.**

## IV · 초대

여러분의 아이디어는  
무엇입니까?

**이제 당신 차례.**

## CHAPTER ONE

### *vibe coding* — 감과 흐름으로 짓는 기술

## 바이브 코딩, 다시 정의하기

### 오해

- "AI한테 시키고 퇴근하는 것"
- "대충 짜는 것"
- "프로 개발자가 아닌 사람의 장난"

### 실제

- **감각과 의도**로 방향을 잡고
- **AI와 실측**으로 속도를 얻고
- **완성된 것**을 남기는 작법

바이브 코딩은 "무엇을 만들고 싶은가"에 집중하기 위해,  
"어떻게 짜는가"의 무게를 AI에게 나눠주는 것.



## 지금이 특별한 이유

- 10년 전 — LLM 추론 엔진 하나 만들려면 **C++·CUDA 전문가 10명**
- 3년 전 — **수백 시간의 논문 독해** + 수개월의 커널 튜닝
- 오늘 — **도메인 감각 + AI 페어 + GPU 한 대로** 시작 가능

**접근 가능성**이 폭발했다.

남은 건 **당신은 무엇을 짓고 싶은가**뿐.

## 바이브 코딩의 네 기둥

### ① 감각 (Taste)

- "이게 맞는 방향인가" 를 느끼는 힘
- 벤치마크 숫자, 코드 냄새, 구조의 아름다움

### ② 흐름 (Flow)

- 멈추지 않는 피드백 루프
- AI와 대화, 즉시 실측, 바로 반영

### ③ 작게 (Small steps)

- 하루 한 세이더, 한 벤치, 한 스파이크
- "완벽한 한 번" 대신 "측정 가능한 스물"

### ④ 공개 (Share)

- 중간 결과를 계속 드러내기
- 피드백이 다음 바이브를 만든다

## 바이브 코딩의 실제 루프

질문 → 초안 → 실행 → 측정 → 의심 → 수정  
그리고 다시 질문.

1. **질문** — “지금 병목은 어디인가?”
2. **초안** — AI에게 가능한 설계와 코드를 뽑아낸다
3. **실행** — 작게 붙여서 바로 돌린다
4. **측정** — 숫자와 로그로 판단한다
5. **의심** — 좋아 보이는 답도 한 번 더 검증한다

핵심은 AI가 아니라, 이 루프를 끊지 않는 것.

## 사람과 AI의 역할 분담

### 사람이 맡는 것

- 목표와 제약 정의
- 성공 기준 결정
- 숫자의 의미 해석
- 위험한 변경의 승인
- 마지막 책임

### AI에게 맡기는 것

- 빠른 초안과 대안 제시
- 생소한 API 탐색
- 반복 코드 생성
- 테스트 아이디어 확장
- 놓친 가정 질문하기

AI는 **가속 페달**입니다.

핸들은 여전히 사람이 잡아야 합니다.

## CHAPTER TWO

### *하나의 증거, 하나의 사례*

## 배경 — LLM 추론 엔진이 뭐길래

학습된 거대 언어모델을 받아, 입력 문장에 대해

다음 토큰을 예측해 내뱉는 **행렬곱** 기계.

- 대표 주자: **llama.cpp · vLLM · Ollama**
- 대부분 **C++ + CUDA** — NVIDIA 전용, 진입 장벽 높음
- **C# + Vulkan**으로 만든 예시? — 찾기 어려웠다

## 질문

그럼 *우리가* 그 공백을 메우면  
어떻게 될까?

— 이게 IgnisCore의 출발점입니다.

# IgnisCore 한 눈에

## 기술 스택

- .NET 10 / C# 14
- Vulkan 1.3 (Silk.NET)
- GLSL → SPIR-V 컴퓨트 셰이더
- Gemma 4 E4B-IT (GGUF Q8\_0)
- NativeAOT 단일 실행파일

## 아키텍처

- C#이 **지휘**하고
- 셰이더가 **연산**한다

## 지금까지 이룬 것

- **decode 75 tok/s** · llama.cpp 대비 **94%**
- **prefill 1,374 tok/s** (pp = 4,096)
- Q8\_0 / BF16 / FP16 전부 지원
- Flash Attention Split-K · CoopMat2
- 멀티턴 대화 · 스트리밍 · Vision
- **CUDA 제로** — 다양한 Vulkan 지원 GPU에서



## 데모에서 볼 것

### 화면으로 확인

- 모델 로딩과 VRAM 사용량
- 프롬프트 prefill 시간
- 토큰 스트리밍 출력
- 멀티턴 대화 유지
- 벤치마크 숫자

### 말로 짚을 것

- “C#이 지휘한다”의 의미
- 셰이더 dispatch가 쌓이는 방식
- 빠른 부분과 아직 느린 부분
- llama.cpp와 비교할 때의 정직한 선

— 데모는 완벽함을 증명하는 시간이 아니라, 실제로 움직인다는 감각을 공유하는 시간.

# 토큰 하나가 지나가는 길

prompt tokens

- embedding
- 42 transformer layers
  - RMSNorm → QKV → RoPE → Attention → MLP
- final norm
- LM head
- sampler
- next token

LLM 엔진은 거대한 마법이 아니라,  
작은 연산을 **정확한 순서**로 수천 번 이어 붙인 것.

## 숫자로 보는 여정

12×

DECODE 성능 향상

6.36 → 75.4 tok/s

94%

LLAMA.CPP 대비

decode tg=128 기준

1,853

GPU 텐서

하나의 세션에서 움직이는

630

DISPATCH / TOKEN

토큰 한 개 = 수백 번의 커널 호출

## 핵심 기술, 세 장면

### VRAM 설계

- 가중치 7.6 GB
- KV 캐시 1.2 GB
- Q8\_0 split + DP4a
- 메모리 대역폭이 전부

### Attention

- Flash Attention Split-K
- score·softmax·value
- **한 셰이더에 융합**
- 긴 컨텍스트에서 버틴다

### TurboQuant

- Walsh-Hadamard 회전
- Lloyd-Max 양자화
- QJL sign 보정
- KV **~4× 압축, cos 0.99+**

— 세부는 이어지는 별도 딥다이브 슬라이드 / 소개 자료 참조.

## 아키텍처를 한 문장으로

C#은 오케스트라 지휘자,  
Vulkan 셰이더는 각 파트의 연주자입니다.

| 계층                | 역할                  |
|-------------------|---------------------|
| CLI / ChatSession | 사용자 입력, 대화 기록, 스트리밍 |
| TransformerEngine | 레이어 순서, KV 캐시, 샘플링  |
| TensorOps         | Vulkan dispatch 래퍼  |

— 복잡한 엔진도 결국 역할을 나누면 사람이 이해할 수 있는 크기가 됩니다.

## 커널 하나가 바꾸는 세계

|        |                             |   |            |          |
|--------|-----------------------------|---|------------|----------|
| 초기     | GPTQ INT8 naive GEMM        | → | 6.36 tok/s |          |
| step 1 | shared-memory tiling        | → | 7.10       |          |
| step 2 | fused RMSNorm / residual    | → | 9.80       |          |
| step 3 | fused attention (decode)    | → | 10.28      |          |
| step 4 | Q8_0 split + DP4a           | → | 48.50      | ← 결정적 점프 |
| step 5 | dual fusion (Q8_1 quant)    | → | 73.96      |          |
| step 6 | flash attn split-K (decode) | → | 74.40      |          |
| step 7 | CM2 L-tile + fp16 acc       | → | 75.40      | ★        |

한 줄 한 줄이 곧 그래프.

**측정 가능한 재미** — 이제 바이브 코딩의 중독성입니다.

## 성능 개선의 실제 패턴

### 처음엔 거의 항상

- 정확하지만 느리다
- dispatch가 너무 많다
- 메모리 왕복이 많다
- 중간 텐서가 쌓인다

최적화는 영감이 아니라 **추적**입니다.  
느린 곳을 찾고, 하나씩 지우는 일.

### 개선은 대개

- 연산을 합친다
- 레이아웃을 바꾼다
- 캐시를 재사용한다
- 측정으로 착각을 지운다

## CHAPTER THREE

*llama.cpp*, 그리고 전문성에 대한 존경



## 정직하게 말하자면

IgnisCore가 **94%** 까지 왔다는 건,  
**llama.cpp가 여전히 선두**라는 뜻입니다.

- prefill pp=512는 여전히 **72% 수준**
- CoopMat2 prefill 이식은 아직 진행 중
- 그 격차 안에는 **수천 시간의 도메인 깊이**가 쌓여 있다

# llama.cpp는 왜 위대한가

## 축적

- 수백 명의 기여자
- 수만 개의 커밋
- 수년간의 벤치마크 데이터
- 수십 개의 백엔드  
(CUDA·Metal·Vulkan·SYCL...)

## 깊이

- 엣지 케이스가 처리된 토크나이저
- 아키텍처별 정교한 커널 분기
- 모델 포맷 호환성의 사실상 표준
- 버그 리포트가 수 시간 안에 수정됨

바이브 코딩으로 **다아갈** 수는 있습니다.

하지만 **그대로 대체하기**란 오랫동안 어려움을 겪니다.

## 전문성은 사라지지 않는다

- AI가 코드 줄을 짜주는 시대에도, **"무엇을 짤지"** 는 사람이 결정합니다
- **도메인 감각**이 없으면 — 벤치마크 숫자의 의미를 읽지 못합니다
- **경험**이 없으면 — 어느 커널이 병목인지 10시간을 헤맵니다
- **이론**이 없으면 — TurboQuant 같은 기법을 **고르는** 것부터 막힙니다

**바이브 코딩은 전문성을 대체하지 않습니다.**  
전문성을 **증폭** 시킬 뿐입니다.

## 초보와 전문가, 각자의 바이브

### 초보의 바이브

- "일단 해본다"의 힘
- 빠른 시작, 빠른 실패
- AI가 제공하는 **진입 경사의 완화**
- 새로운 조합을 두려워하지 않음

### 전문가의 바이브

- "어디가 진짜 병목인지" 감지
- 벤치 숫자에서 **이야기**를 읽음
- 퓨전·양자화·타일링의 정교한 선택
- 한 줄 수정이 30% 개선으로

— 두 바이브는 경쟁이 아니라 **협업**합니다.

## IgnisCore가 증명한 것, 증명하지 않은 것

### 증명한 것 ✓

- C#만으로 **LLM 엔진이 가능**하다
- 개인 한 명이 **실용 수준**까지 갈 수 있다
- CUDA 없이 **크로스 GPU** 추론이 된다
- NativeAOT로 **단일 바이너리 배포** 가능

### 증명하지 않은 것 ✕

- llama.cpp를 **대체**한다 — 아니다
- 모든 모델에서 **제일 빠르다** — 아니다
- 프로덕션에서 **가장 안정**하다 — 아니다

바이브 코딩의 결과를 **정직하게** 평가하는 것,  
그 자체가 또 하나의 바이브입니다.

## CHAPTER FOUR

*무엇을, 왜, 어떻게 지을 것인가*

---

오늘 가장 하고 싶은 말

여러분도 할 수 있습니다.

그리고 **해본 적 없던** 분야에서 더욱.

- LLM 엔진만 재미있는 게 아닙니다
- 바이브 코딩이 열어주는 **새로운 놀이터**가 있습니다

# 바이브 코딩으로 가볼 만한 땅들

## 시스템 · 저수준

- 자기만의 **VM / 인터프리터**
- 미니 **OS 커널**
- **쉐이더 기반 물리엔진**
- **데이터베이스 엔진** (WAL · B-Tree)
- 가볍게 쓰는 **컴파일러 프론트 엔드**

## AI · 수치

- **RAG / 벡터 DB** 실험
- 소형 모델 **로컬 파인튜닝**
- **KV 압축 · 스파스 어텐션**
- 오디오 · 이미지 **확산 모델**
- 나만의 **임베딩 파이프라인**

## 창작 · 삶

- 조명 / 사운드 **인터랙티브 설치**
- **게임 엔진 마이크로코어**
- 개인 **음악 제너레이터**
- 하드웨어 연계 **스마트 오브젝트**
- **생활 자동화** — 가계부 · 일기 AI

— 공통점: **"전엔 너무 커 보여서 포기했던 것들."**



## 아이디어를 고르는 법

1. **오래 품고 있던 것** — "언젠가는..." 리스트에서 가장 위
2. **개인적 집착** — 남들이 이해 못 해도 **내가** 재미있는
3. **작게 쪼갤 수 있는 것** — 한 주에 한 조각씩 굴릴 수 있는
4. **측정 가능한 것** — 숫자든, 소리든, 시각이든 피드백이 즉각적인

세상이 **원하는 것** 이 아니라,  
당신이 **보고 싶은 것** 을 고르세요.  
바이브는 거기서만 나옵니다.

## 시작을 위한 일곱 걸음

1. **한 문장으로** 무엇을 만들고 싶은지 적는다
2. **30분**짜리 첫 스파이크 — "Hello, 무엇"
3. AI에게 **구조**를 물어보고 — 전체는 직접 설계한다
4. 매일 **하나의 측정 가능한 진전** — 숫자든 스크린샷이든
5. **공개**한다 — 블로그, 리포, 디스코드, 어디든
6. 막히면 **아래층** 으로 내려간다 — 한 추상화 더 낮게
7. **즐거움이 식으면 쏜다** — 바이브는 억지로 안 된다

## 바이브 코딩의 원칙 (재배치)

### 해야 할 것

- 작게 시작하라
- 자주 측정하라
- 정직하게 실패하라
- AI와 대화하되 결정은 내가
- 자신의 취향을 믿어라

### 하지 말아야 할 것

- AI에게 전부 맡기기
- 벤치마크 숫자를 꾸며내기
- 도메인 지식을 생략하기
- 완성 전에 지치도록 몰기
- 남의 로드맵을 복사하기

## 실패도 바이브다

- 세이더 하나가 일주일 내내 검은 화면만 출력한 적이 있습니다
- `PipelineBarrier` 하나 빠뜨려서 **모델이 이상한 언어로 말한** 적도
- KV 레이아웃 실수로 **멀티턴 대화가 무너진** 적도

그 모든 실패가, 결국 **다음 커밋의 재료**였습니다.

— 기록하세요. 모든 실패는 재산입니다.

## 실패를 다루는 루틴

1. **재현** — 같은 입력에서 같은 문제가 나는지 확인
2. **격리** — 셰이더, 버퍼, 샘플러, 토크나이저 중 어디인지 좁히기
3. **참조값** — CPU 또는 llama.cpp와 비교할 작은 케이스 만들기
4. **기록** — 원인·증상·해결책을 남기기
5. **회귀 방지** — 다음에는 자동으로 잡히게 만들기

바이브는 감각이지만,  
디버깅은 **절차**입니다.

## 바이브 코딩을 지탱하는 도구들

| 무엇을   | 왜               | 대표 도구                     |
|-------|-----------------|---------------------------|
| AI 페어 | 탐색 · 초안 생성 · 설명 | Copilot / Claude / Cursor |
| 버전 관리 | 돌아올 지점 보장       | Git + GitHub              |
| 측정    | 바이브를 숫자로        | 벤치 스크립트 · 프로파일러           |
| 공유    | 다음 바이브의 연료      | 블로그 · 리포 · 데모 영상          |
| 기록    | 미래의 나에게 주는 선물   | 개발 일지 · 메모리 파일            |

도구는 많을수록 좋은 게 아닙니다. **자기에게 맞는 것만.**

## 마지막으로 전하고 싶은 것

- IgnisCore는 "대단한 엔진" 이 아닙니다. 그냥 **한 사람의 바이브가 만든 것** 입니다
- llama.cpp는 여전히 선두이고, 전문성은 여전히 빛납니다
- 그리고 — **이 두 사실은 서로 모순되지 않습니다**

당신이 지금 **너무 크다**고 느끼는 그 아이디어.  
올해, 바이브 코딩으로 **시작**해보세요.

여러분의 다음 토큰은  
무엇입니까?



*Vibe on. Burn bright.*

[github.com/dimohy/IgnisCore](https://github.com/dimohy/IgnisCore)

Q&A

LET'S VIBE